

## Guia de trabajos prácticos N° 4

### Microprogramación

1. Codificar las operaciones en microinstrucciones para la arquitectura horizontal de Tanenbaum:
  - a.  $Ax = (Ax + Bx) * 2$
  - b.  $Dx = (Fx + Ax) / 4$
  - c.  $[Fx] = [Cx - Dx]$
  - d.  $[(IR \text{ and } AMask)] = Ax$
  - e.  $Bx = Ex \text{ AND } [Fx]$
  - f.  $[SP+1] = [Dx * 2] + Cx$
  - g.  $Ax = Ax \text{ or } Bx$
  - h.  $PC = [Dx] + Cx$
  - i.  $[(IR \text{ and } AMask)] = (Ex * 2) + (Ex / 2)$
  - j.  $[Dx] = (Ax + Bx + Cx) * \frac{1}{2}$
  - k.  $Ax = Ax * Bx$
  - l.  $Ax = Ax / Bx$
2. Resolver las operaciones del ejercicio anterior usando el conjunto de microinstrucciones de la arquitectura vertical de Tanenbaum.
3. Escribir rutinas en **microinstrucciones horizontales y verticales** para extender el conjunto de instrucciones de assembler. Para cada una de ellas, se debe agregar código al microprograma comenzando en la dirección 0x60 la memoria de control. Se asume que el microprograma ya decodificó la instrucción que se encuentra en el registro IR. Al finalizar el microcódigo, utilizar como retorno la dirección 0x00. Las instrucciones **no deben modificar** los registros de propósito general, a menos que esté expresamente especificado.
  - a. **INV** (Invert): **cambia el signo del valor de AC, en complemento a dos.**
  - b. **TABS** (Top Absolute Value): **pone en valor absoluto el valor al tope de la pila.**
  - c. **ANDI** (Indirect And): **realiza el AND entre AC y el valor de la memoria apuntado** por el operando de la instrucción. La instrucción de 16 bits tiene el código de operación en los 4 bits más significativos y el operando en los siguientes 12 bits.
  - d. **MULD** (Direct Mult): **multiplica el valor del operando por AC** y deja el resultado en AC. El operando se encuentra en los 12 bits menos significativos de la instrucción.
  - e. **DIVI** (Indirect Div): **divide AC por el valor en la memoria apuntado por el operando** (12 bits) y deja el resultado de la división en AC y el resto en AC.
  - f. **JTE** (Jump Top Equal): **salta a la dirección del operando, si el tope de la pila es igual a AC.**
  - g. **BSR** (Bit Scan Reverse): **busca el bit 1 menos significativo** de la celda de memoria que se encuentra al tope de la pila **y devuelve su índice en AC**. En caso que el tope de la pila contenga 0x0000, deja en AC el valor -1.  
**Ejemplos:** 0x0005 → AC=0, 0xFFFF0 → AC=4
  - h. **BSF** (Bit Scan Forward): lee el valor apuntado por su operando (12 bits) y **guarda el índice del primer "1"** en AC. En caso de ser cero, deja -1 en AC.  
**Ejemplos:** 0x0008 → AC=3, 0x1245 → AC=0, 0x0000 → AC=-1
  - i. **BLSR** (Bitwise Reset): **apaga (pone en 0) el bit 1 menos significativo**, dejando el resto intacto, de la celda de memoria en la posición de memoria indicada en su único operando (12 bits). Es el equivalente por hardware a hacer  $x = x \& (x - 1)$  en C.
  - j. **POPCNT** (Population Count): cuenta el número de bits establecidos en 1 (bits activos) dentro del registro AC y lo guarda en la posición de memoria indicada en su único operando (12 bits).  
**Ejemplos:** AC=0x0005 → 0x0002, AC=0x0100 → 0x0001, AC=0xFFFF → 0x0010

4. Dada la arquitectura vertical propuesta por Tanenbaum y su matriz representativa del decodificador de operación, se quieren implementar nuevas microinstrucciones, a saber:

a. **SetMBR** carga el registro MBR con el contenido de un registro de uso general,  $MBR:=r2$

Ejemplo de uso: *SetMBR* ,AC

b. **AddMBR** acumula el contenido del registro MBR y de un registro de uso general,  $r1:=MBR+r1$

Ejemplo de uso: *AddMBR* AX

c. **ZJMBR** si el valor del registro MBR es cero, entonces salta a la microinstrucción cuya dirección se calcula como  $(r1 < 4) \mid r2$

Ejemplo de uso: *ZJMBR* 0x1, 0x6

| Código de<br>operación de<br>microinstrucción | A<br>L<br>U<br>H | A<br>L<br>U<br>L | S<br>H<br>H | S<br>H<br>L | N<br>Z | A<br>M<br>U<br>X | A<br>N<br>D | M<br>A<br>R | M<br>B<br>R | R<br>D | W<br>R | M<br>S<br>L<br>H | M<br>S<br>L<br>L |
|-----------------------------------------------|------------------|------------------|-------------|-------------|--------|------------------|-------------|-------------|-------------|--------|--------|------------------|------------------|
|-----------------------------------------------|------------------|------------------|-------------|-------------|--------|------------------|-------------|-------------|-------------|--------|--------|------------------|------------------|

|    |               |   |   |   |   |   |   |   |   |   |   |   |   |
|----|---------------|---|---|---|---|---|---|---|---|---|---|---|---|
| 0  | ADD           |   |   |   |   | + |   | + |   |   |   |   |   |
| 1  | AND           |   | + |   |   | + |   | + |   |   |   |   |   |
| 2  | MOVE          | + |   |   |   | + |   | + |   |   |   |   |   |
| 3  | COMPL         | + | + |   |   | + |   | + |   |   |   |   |   |
| 4  | LSHIFT        | + |   | + |   |   |   | + |   |   |   |   |   |
| 5  | RSHIFT        | + |   |   | + |   |   | + |   |   |   |   |   |
| 6  | GETMBR        | + |   |   |   | + | + | + |   |   |   |   |   |
| 7  | TEST          | + |   |   |   | + |   |   |   |   |   |   |   |
| 8  | BEGRD         | + |   |   |   |   |   |   | + |   | + |   |   |
| 9  | BEGWR         | + |   |   |   |   |   |   | + | + |   | + |   |
| 10 | CONRD         | + |   |   |   |   |   |   |   |   | + |   |   |
| 11 | CONWR         | + |   |   |   |   |   |   |   |   |   | + |   |
| 12 |               |   |   |   |   |   |   |   |   |   |   |   |   |
| 13 | NJUMP         | + |   |   |   |   |   |   |   |   |   |   | + |
| 14 | ZJUMP         | + |   |   |   |   |   |   |   |   |   | + |   |
| 15 | UJUMP         | + |   |   |   |   |   |   |   |   |   | + | + |
| 12 | <b>SetMBR</b> |   |   |   |   |   |   |   |   |   |   |   |   |
| 12 | <b>AddMBR</b> |   |   |   |   |   |   |   |   |   |   |   |   |
| 12 | <b>ZJMBR</b>  |   |   |   |   |   |   |   |   |   |   |   |   |

Si pudiera agregar al diseño del decodificador de operación una de estas tres instrucciones con código 12, ¿cuál agregaría y por qué?